SLIDING WINDOW + TWO POINTERS

FAANG revision cheat sheet | 2-3 minute review | One-page decision flow

01 IDENTIFY THE PATTERN

Contiguous subarray / substring? Consider sliding window. **Not contiguous?** Consider two pointers or other techniques.

02 CHOOSE YOUR TECHNIQUE

Window of size k - Fixed sliding window

Longest / maximum valid window - Variable window; shrink when invalid

Shortest / minimum valid window - Variable window; shrink while valid

Count subarrays with at most k - Variable window + count valid endings

Count subarrays with exactly k - AtMost(k) - AtMost(k-1), when applicable

Pair in sorted array - Opposite-direction two pointers

Remove duplicates / partition - Same-direction two pointers

Arbitrary integers, target sum - Prefix sum + HashMap

03 CHECK MONOTONICITY

Ask: Can extending or shrinking the window change validity predictably? The **condition** must be monotonic, not necessarily the original array.

YES - Sum of nonnegative numbers <= k

YES - At most k distinct characters

YES - At most k odd numbers (even for negative inputs)

USUALLY NO - Sum <= k with negative numbers

YES - Fixed-size windows, including negative values

Golden rule: For variable-size counting, once a start index is discarded, later expansion must not make it relevant again.

04 TWO POINTER DECISION RULES

Opposite directions - left=0, right=n-1: Two Sum II, Container With Most Water

Same direction - Slow/fast: remove duplicates, move zeroes, in-place modification

Expand / shrink - Longest substring, minimum window, valid-subarray counting

05 CODE TEMPLATES

A. Longest valid window - shrink when invalid

```
for(int right=0; right<n; right++) {
    add(nums[right]);
    while(invalid()) {
        remove(nums[left]); left++;
    }
    ans = max(ans, right-left+1);
}
```

B. Shortest valid window - shrink while valid

```
for(int right=0; right<n; right++) {
    add(nums[right]);
    while(valid()) {
        ans = min(ans, right-left+1);
        remove(nums[left]); left++;
    }
}
```

C. Count at most k - count valid endings

```
for(int right=0; right<n; right++) {
    add(nums[right]);
    while(invalid()) {
        remove(nums[left]); left++;
    }
    ans += right-left+1;
}
```

06 COUNTING FORMULAS

At most k: `ans += right - left + 1` - all suffixes of the valid window ending at **right** are valid.

Exactly k: `AtMost(k) - AtMost(k-1)` - for monotonic integer counts (odds, distinct, nonnegative sums).

Prefix sum + HashMap: `previousPrefix = currentPrefix - k`. Works with negative numbers. Initialize **mp[0]=1**, query before inserting current prefix.

07 FINAL 5-SECOND CHECKLIST

[ ] 1 - Contiguous window or sorted pair?

[ ] 2 - Fixed or variable size?

[ ] 3 - What is valid / invalid? Is validity monotonic?

[ ] 4 - Shrink while valid or when invalid?

[ ] 5 - Maximize, minimize, or count? Does formula apply?

Interview traps: while vs **if**; **ans++** vs **ans += right-left+1**; assuming Exactly(k) always needs a HashMap.